

ResumeIQ — Q&A Prep Sheet

PRESENTATION REFERENCE · 4 SECTIONS · 33 QUESTIONS

A · ABOUT THIS PROJECT

Q Why Groq instead of OpenAI or Gemini?

Free-tier access to strong open-weight models via an OpenAI-compatible API — a drop-in integration, and fast (low inference latency). Avoids locking into one closed provider.

Q Why embed resumes locally instead of using an embeddings API?

Avoids extra API cost and rate limits, works offline, and keeps resume text from going to a second third-party service. Every resume needs embedding, so a small local model (~90MB) is fast and free at this scale.

Q Why not use a real vector database like Pinecone or Chroma?

At hundreds of resumes (not millions), brute-force cosine similarity runs in under a millisecond with zero extra infrastructure. A real vector DB earns its keep past thousands of candidates — a legitimate next step, not a current need.

Q How do you stop the AI from hallucinating skills or scores?

The prompt grounds the model in the actual resume text and instructs it to be realistic and differentiated, not generically high. Not foolproof, but it's reading real text, not recalling training knowledge.

B · ALTERNATIVES & COMPARISONS

Q How is this different from just pasting resumes into ChatGPT?

A chatbot has no memory of your candidate pool and nothing persistent to search. This keeps a running, searchable record of every candidate ever screened, specific to your own pipeline.

Q Why not use an ATS platform with built-in AI screening?

Enterprise ATS tools are heavyweight, often expensive, and their scoring is a black box. This is transparent — you see exactly why a score was given and exactly which candidates were retrieved.

Q Could you do this with keyword search instead of embeddings?

Keyword search misses meaning — it wouldn't connect "led cloud infrastructure migrations" to a search for "AWS experience." Embeddings match by concept, not exact words.

C · FUTURE ENHANCEMENTS & SCALING

Q How would you scale this to millions of resumes?

Replace brute-force similarity with a vector database using an approximate-nearest-neighbor index, and move storage off a single JSON file onto a proper database.

Q How would you add multi-user or team support?

Add accounts, scope every session and embedding to a user/organization ID, and enforce access control at the database level. Currently single-tenant.

Q How would you improve accuracy on very long resumes?

Chunk each resume into sections (experience, education, skills), embed each chunk separately, and retrieve at the chunk level — the standard RAG chunking pattern.

D · CORE AI / ML / RAG CONCEPTS

Q What happens if the AI returns malformed output?

The response is parsed as JSON after stripping markdown fences; if parsing fails, that one resume gets a 0% score and an "analysis failed" note instead of crashing the whole batch.

Q How does this behave on serverless hosting, where local storage doesn't persist?

It tests filesystem writability at startup; if read-only, it automatically switches to an in-memory store synced with a small cloud key-value service — same code, no config change.

Q How do you handle privacy, since resumes contain personal data?

Data stays local or in a private cloud store scoped to your own bucket ID. Resume text is sent to Groq for scoring — inherent to using any hosted LLM. (No automatic deletion or encryption-at-rest yet — an honest current gap.)

Q Why RAG instead of fine-tuning a model on resume data?

Fine-tuning needs a large labeled dataset and real training infrastructure, and must be redone whenever data changes. RAG works instantly on whatever resumes exist right now, at far lower cost.

Q Why not just paste every resume into one giant prompt instead of retrieving?

Doesn't scale — cost and latency grow with every resume added, and models perform worse when relevant information is buried in a huge prompt. Retrieval keeps every query fast regardless of scale.

Q How would you address bias in AI resume scoring?

Strip identifying details like name and photo before scoring, periodically audit score distributions for skew, and keep a human in the loop for final decisions.

Q How would you measure whether the scoring is actually accurate?

Build a labeled test set with human-assigned scores, run it through the pipeline, measure agreement, and let recruiters flag right/wrong scores to build a real evaluation dataset.

Q What is RAG, in one sentence?

A technique where the model retrieves the most relevant information from a knowledge base before generating its answer, instead of relying only on training memory.

Q What's the difference between an embedding and a normal search index?

A text index matches literal words; an embedding is a vector representing meaning, so it matches by concept even with completely different wording.

Q What is cosine similarity, simply?

A measure of how similar the "direction" of two vectors is — 1 means identical meaning, 0 means unrelated, regardless of vector length.

Q What does "384 dimensions" actually mean?

Each piece of text becomes a list of 384 numbers. No single number is meaningful alone, but together they place similar texts close together in a 384-dimensional space.

Q What's the difference between a language model and an embedding model?

A language model generates text by predicting the next word. An embedding model produces a fixed-size vector representing meaning — it compares, it doesn't generate.

Q Is this actually machine learning, or just code?

Both. The embedding model and language model are trained neural networks — that's the ML. Cosine similarity is plain math operating on outputs from a trained model.

Q What is a hallucination, and how does RAG reduce it?

A confident but false statement, since the model predicts plausible text rather than looking anything up. RAG grounds answers in real retrieved documents, sharply reducing — not eliminating — that risk.

Q What is a "context window"?

The maximum text a model can consider in one request. RAG's whole purpose is fitting only the relevant few documents into that limited space instead of everything.

Q What is a "token"?

The unit of text a model actually processes, roughly a word or part of a word. API costs, rate limits, and the context window are all measured in tokens.

Q What is prompt engineering?

Carefully wording instructions to a model to get a reliable, structured output — e.g. requiring valid JSON in a specific schema, or asking it to be critical rather than generically positive.

Q What's self-supervised learning, and why does it matter for embeddings?

Training on unlabeled text by having the model learn structure from the text itself (e.g. pulling similar sentences together in vector space) — no manual labeling needed, which is how embedding models are built at scale.
